
Labwork

SIGNAL PROCESSING 2

ADVANCED SPECTRAL ANALYSIS COURSE
SPECIALITE PHYSIQUE
DEUXIEME ANNEE



CentraleSupélec

Supervising teacher
STEPHANE ROSSIGNOL
stephane.rossignol@centralesupelec.com

Abstract

You are expected to be inventive and self-reliant. Describe your ideas and tips in as much detail as possible. Try to make these tips as universal as possible (keep in mind, though, that absolute universality is unattainable).

Aim to write code that is clean, readable, and understandable by someone completely new to the problem ; someone who encounters it unexpectedly and needs to become productive as quickly as possible.

Write your report (in preparation for the evaluation) without delay. You will save time this way.

Contents

1	Detection of fundamental frequency	3
2	Filter banks	3
3	Wavelets	4
3.1	Denoising signal using wavelets	4
3.2	Source separation	4
4	MP/OMP/BP	5

1 Detection of fundamental frequency

In this part, you will implement algorithms to detect f_0 on the signals “fluteircam.wav” and “voiceP.wav”.

Non-parametric spectral analysis method used : the FFT. Start by testing this method. Ground truth data for both files is provided : see the files “verite-terrain-flute.txt” and “veriteterrainvoiceP.txt”. Tasks to be performed :

- detection of f_0 using the FFT (every 10 ms)
- silence detection using autocorrelation and/or energy
- detection of f_0 using autocorrelation (every 10 ms)
- comparison of f_0 via FFT vs. f_0 via autocorrelation. Performance metrics : the percentage of audio frames, out of the total number of frames, for which the correct f_0 is found

Then, potentially, to complement these analyses :

- tempo detection (quarter note/half note/whole note)
- vibrato analysis (frequency and amplitude)

Once the lecture on parametric/advanced methods has begun, replace the FFT with one of these methods and repeat the procedure described above (then test other methods ; and, for instance, ultimately consider the following subset of methods : autocorrelation/FFT/Burg/MUSIC). Note : you will need to limit yourself to a subset of methods.

2 Filter banks

The problem statement, covered in class (and corrected in class), reads as follows : we have a harmonic signal composed of the sum of four sine waves. The fundamental frequency is 880 Hz, and the sampling frequency is 8000 Hz.

How many successive decompositions are required to recover at most one sine wave per band ?

So, to which frequencies were the four initial sine waves transposed ?

Which method(s) should be used to find the frequencies of these four sine waves ?

Question 1 Carry out this exercise in practice (using the language of your choice : Matlab, Octave, or Python). To begin, generate the corresponding sound (it can last about a second or longer ; however, note that the longer it is, the more time the analysis will take). Then, save the sound to a .wav file (using “audiowrite” in Octave/Matlab). Listen to it to verify that everything is correct (using Audacity). Finally, analyze the sound as described in the exercise.

Question 2 Add a vibrato (frequency modulation) with an amplitude of 20 Hz and a frequency of 5 Hz to the original signal, and comment on the result. Note : when calculating the spectrogram (or scalogram), for instance, ensure that you have actually added the intended vibrato ; some equations set up are required to correctly implement the specific vibrato. Then, gradually increase the vibrato amplitude and comment on the effect.

Help center See Python libraries such as numpy, matplotlib, scipy.signal. Also, see : <https://www.f-legrand.fr/scidoc/docimg/sciphys/caneurosmart/pysignal/pysignal.html>

3 Wavelets

3.1 Denoising signal using wavelets

To begin with, try to reproduce the example given in class. You need to implement a measure of the quality of the denoising achieved : SNR before denoising and SNR after denoising.

Then, try using several types of wavelets. It may also be interesting to vary the number of decompositions.

Experiment with the other available parameters (threshold, noise variance, etc.) to observe and effectively measure the effects of each one.

Try setting the threshold automatically : Donoho’s universal threshold, for example, is commonly used in image processing, etc. (there are dozens of automatic thresholds).

3.2 Source separation

Next, I propose performing source separation using wavelets. Note : here, we are dealing with a challenging scenario : we have fewer sensors than sources, since there is one sensor and two sources.

To begin with, try to reproduce the example given in class. The quality of the obtained source separation must be characterized.

Then, try using several types of wavelets. Play around with the number of decompositions. Experiment with the other available parameters (threshold, frequencies of the two sine waves, etc.) to observe and actually measure the effects of each one. Try to automatically determine the threshold.

4 MP/OMP/BP

The idea is to perform audio compression using “pursuit” techniques. The primary challenge (and the most important one) is, of course, to achieve the highest possible level of sparsity (i.e., the greatest compression ratio) ; however, one must keep in mind that computation time is also a crucial factor (for instance, telephony applications require real-time processing, meaning that less than one second of computation time is available to process one second of audio).

Compare the performance of MP, OMP, and BP on a sound recording you have made say a short phrase (just a few words), paying close attention to the recording quality (avoiding clipping and maximizing dynamic range), but not too long (2 or 3 seconds), otherwise the computation time may become excessive (depending on the computer and the techniques used).

What needs to be compared :

- the final errors for the three types of “pursuit” (the SNRs)
- the computation times for the three types of “pursuit”
- the compression ratios obtained for the three types of “pursuit”

Note : Use the same “frames” for all three techniques to ensure a valid comparison. Try other “frames” besides Gabor plus Hanning (as I did). Concatenate various types of “frames” : STFTs, several types of wavelets, several wavelet “sizes”, and so forth.

Pay attention to the sampling rate (44100 Hz is not necessary for speech ; 32000 Hz or even 16000 Hz is sufficient).

There are plenty of other parameters to configure or test : consult the documentation for the various available functions. For example, there are three algorithms for computing Orthogonal Matching Pursuit (OMP) that do not run at the same speed ; try measuring this as well. Check the “franamp” documentation : look for ‘qr’ (QR-based method), ‘chol’ (Cholesky-based method), ‘cg’ (Conjugate Gradient Pursuit), and ‘auto’ (where “franamp” selects the algorithm).